Flowl - Advanced Overlay Natural Language Processing Real-Time Speech Translator Application Design Document

Version: 1.0
Date: October 23, 2025
Developer: Nikita Petrucek 20248680
Project Type: AI/NLP Real-Time Translation System
Target Platform: Desktop Application

- **Title Page**

- Application name: Flowl
- Application Title: Advanced Real-Time Speech Overlay Caption Translator
- Version: 1.0
- Date: 23/10/2025
- Developer Nikita Petrucek
- Project Type: AI/NLP Systems Programming Application
- Target Platform: Desktop application

2. Table of Contents

1. Title Page
2. Table of Contents
3. Introduction/Overview
4. Application Concept
5. UX and UI
6. Core Processing Modules
7. Visual Design
8. Audio Processing Design
9. Initialization & Setup
10. System architecture and data flow
11. Current Stage of Development & Future planning
12. Possible Features & Concepts

3. Introduction/Overview

Purpose

The application specializes in providing subtitle-style translations that appear over existing content, making foreign language websites, videos, and applications instantly accessible. By combining efficient speech recognition with smart translation algorithms, it enables users to understand content in real-time without disrupting their workflow or requiring separate translation windows. The system's modular architecture ensures easy maintenance and updates while keeping the core functionality lightweight and responsive.

Scope

- Capture live audio from microphone using sounddevice with automatic device detection and fallback mechanisms
- Stream audio data continuously through thread-safe queues with configurable buffer sizes and throttling
- Perform real-time speech-to-text processing using Vosk ASR with dual-mode recognition (partial/final results)
- Filter and segment recognized text using intelligent throttling, minimum thresholds, and duplicate detection
- Queue translation events with thread-safe communication between ASR and MT workers
- Translate text incrementally using Helsinki-NLP transformer models with GPU acceleration and caching
- Process both partial and final results with smart text slicing and efficient memory management
- Display real-time translations through PySide6 GUI with thread-safe UI updates and live subtitle display
- Provide comprehensive logging and configuration with categorized logging, JSON settings persistence, and runtime adaptation capabilities

Target Audience of Users

The Advanced Real-Time Overlay Translator serves desktop users who need quick, unobtrusive translation assistance during their daily computer activities:

Web Browser Users

Individuals browsing foreign language websites, reading international news, or accessing global e-commerce platforms benefit from real-time subtitle overlays that translate content without requiring page navigation or separate translation tools. The application seamlessly integrates with popular browsers to provide instant comprehension of web content.

Media Consumers and Streamers

Users watching foreign language videos on platforms like YouTube, Netflix, or streaming services can access real-time subtitles in their preferred language. The overlay functionality works across different media players and web-based video platforms, enhancing entertainment and educational content consumption.

Remote Workers and Students

Professionals participating in international video conferences, webinars, or online courses utilize the application for real-time translation during virtual meetings. Students accessing foreign language educational content or participating in international online programs benefit from immediate translation support without disrupting their learning environment.

Gamers

Players engaging with international teammates or opponents can use the translator to instantly understand and respond during voice or text-based communication. This feature enhances real-time coordination and fosters cross-cultural connections in multiplayer games.

Casual Desktop Users

Everyday computer users who occasionally encounter foreign language content during routine desktop activities appreciate the lightweight, always-available translation capability. The application runs quietly in the background, activating only when needed, making it ideal for users who want translation support without system overhead.

Privacy-Conscious Users

The application is also well-suited for those who value privacy, as it can operate fully offline, ensuring that sensitive data and communications remain local to the user's device.

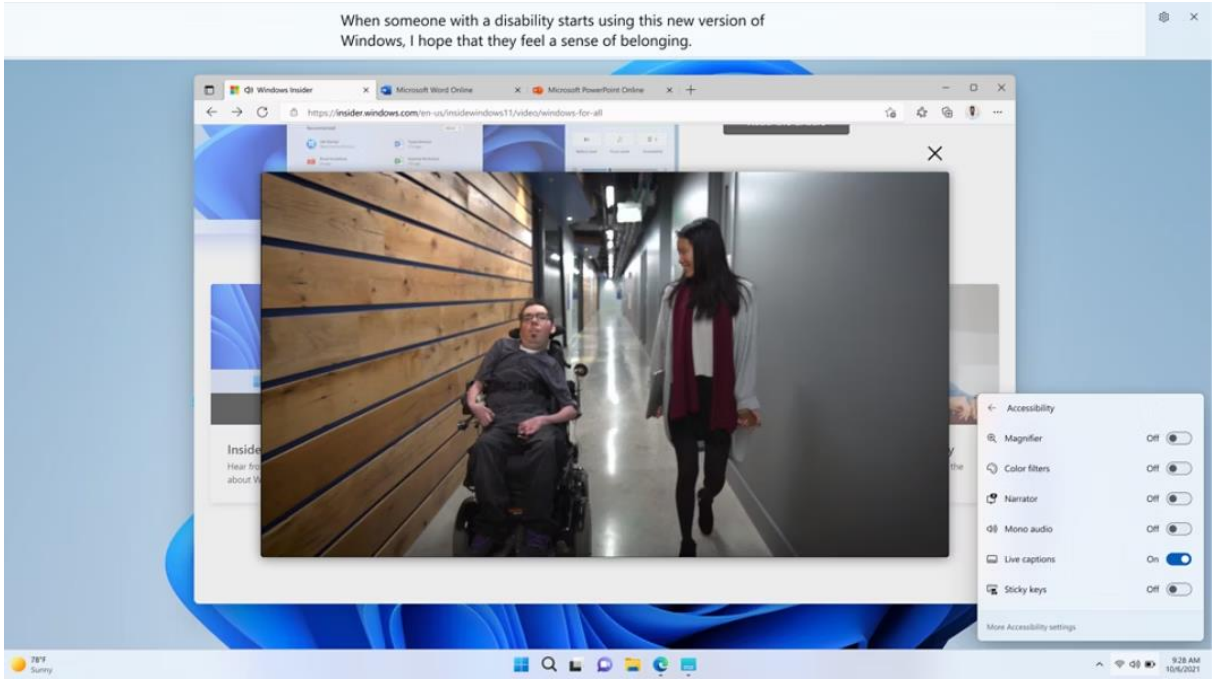
3.1. Application Overview

Flowl is a desktop-focused real-time translation application that provides immediate translation for computer interactions. This system uses neural machine translation models (Helsinki-NLP transformers) with GPU acceleration to deliver accurate translations through a multi-threaded pipeline. It serves as a practical tool for breaking language barriers during web browsing, gaming, video streaming, and online meetings, displaying translations in a dedicated PySide6 window with comprehensive logging and configuration capabilities.

4. Application Concept

Inspirations:

Microsoft Live Captions



Live Captions in Windows 11 automatically convert any system audio (videos, voice chats, or microphone input) into text captions on-screen.

Languages: Translates over 40 languages into English and 27 languages into Simplified Chinese on Copilot+ PCs running Windows 11 version 24H2 or later.

Offline processing: Works entirely on-device; no cloud connection or data sharing is required.

System requirements: Needs advanced AI-capable processors (AMD Ryzen AI 300, Intel Core Ultra 200V, or Snapdragon X series) with at least 40 TOPS NPU performance.

Customization: Users can adjust text style, background color, and window placement (above, below, or floating on screen).

While UI/UX will be mostly similar, mine application will focus on cross-platforming, multiple language pipeline options (not limited to Chinese and English) and not reliant on AI-capable processors.

Key Objectives

- Real-time processing: Translate and display results with minimal delay using multi-threaded pipeline and intelligent throttling
- Neural translation accuracy: Leverage state-of-the-art transformer models (Helsinki-NLP) with GPU acceleration for high-quality translations
- Resource optimization: Use efficient caching, optimized inference parameters, and device management to balance performance with accuracy
- Language pair flexibility: Support multiple language combinations (FROM↔TO) with automatic model selection and configurable settings
- Cross-platform compatibility: Desktop application with PySide6 GUI that works across Windows, macOS, and Linux systems
- Modular architecture: Clear separation of audio capture (AudioEngine), ASR processing (ASRWorker), translation (MTWorker), and UI components with thread-safe communication

- User experience: Intuitive settings management, comprehensive logging, and real-time feedback through dedicated translation window
- System integration: Automatic device detection, fallback mechanisms, and graceful error handling for robust operation
- Performance monitoring: Built-in logging system with categorized output and performance tracking for system optimization
- Configuration management: JSON-based settings with runtime updates and persistent storage for user preferences
- Overlay output: Caption overlay based output similar to Live Captions.

4.1. Core Functionality and Innovation

The application's core innovation lies in its real-time processing pipeline, which uses intelligent throttling and text filtering to provide both partial and final translation results. This approach eliminates traditional delays by translating speech segments as they're recognized, enabling more responsive conversational dynamics through a multi-threaded architecture with queue-based communication between ASR and translation components.

Key features include:

- Real-time Audio Capture and Processing: Continuous microphone input with intelligent segmentation
- Automatic Speech Recognition (ASR): Offline speech-to-text using Vosk models
- Machine Translation: Neural machine translation via Transformers library
- Device Selection and Management: Flexible audio device enumeration with fallback mechanisms
- Settings Configuration UI: Comprehensive multi-tab configuration interface
- Color-coded Logging System: Real-time diagnostic capabilities with visual feedback
- Translation Caching: LRU cache (100 items) for performance optimization
- GPU Acceleration Support: Optional CUDA acceleration for translation models
- Thread-safe Operations: Robust multi-threaded architecture with queue-based communication

4.2 Use Cases / User Stories

• Live conversation assistance: A user speaks in language A; the other party sees subtitles in language B on screen or hears audio via headphones

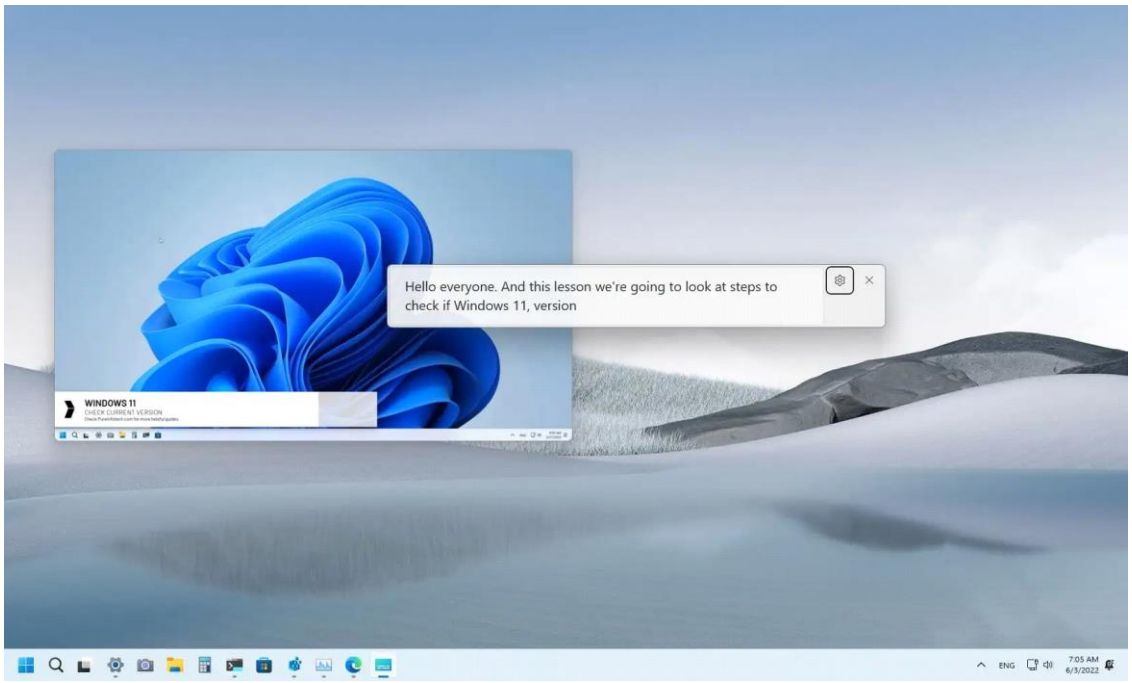
• Lecture or presentation translation: Speaker's words are transcribed and translated in real time for an audience

• Video overlay: The application hooks into audio or video streams and shows translated subtitles on top of video playback

• Offline/edge use: In contexts without reliable internet, the app runs locally using lightweight versions of models and pipeline-based translation logic

• Customization/domain-specific tweaks: Users can adjust models and application speeds and timings according to their needs

5. UX and UI



An example of how final UI might look like

User Interface Goals

The interface design prioritizes minimal distraction while maintaining full functionality. Key design principles include:

• Minimal distraction: Overlay subtitles should be legible but not obstruct critical screen areas
• Extensive customization: Font size, color, position, opacity, background shading

options
• Intuitive feedback & controls for start/stop capture, language selection, and settings adjustment
• Real-time diagnostic capabilities with confidence indicators and processing status
• Accessibility features including keyboard shortcuts, adjustable text size, and color-blind-friendly palettes

Primary Interface Components

- Main overlay/subtitle area: Translucent panel typically positioned at selected position of the screen displaying translated text
- Control toolbar: Minimizable interface with essential controls (start/stop, settings, language selection)
- Logging tab: Provided logging info about app performance
- Settings dialog: Comprehensive configuration interface with tabs for Audio, Translation and Display
- Notification system: Visual cues for translation events, latency alerts, and system status

6. Core Processing Modules

The backend architecture is organized into clear, modular components. Each module exposes a defined interface and communicates via structured data formats. This modular approach ensures maintainability, testability, and extensibility.

Audio Capture & Buffer Controller

Responsibilities: Capture live audio from microphone; maintain thread-safe audio queues; provide continuous audio streaming to ASR processing.

- Inputs: Raw audio stream from selected input device via sounddevice
- Outputs: Audio data (bytes) pushed to thread-safe audio queue
- Features: Device enumeration, automatic fallback, configurable sample rates (16kHz), buffer management
- Interface: AudioEngine with callback system, DeviceManager for device selection
- Current Implementation: Uses sounddevice with 2048 frame buffers, thread-safe deque with maxlen=50

ASR Engine (Vosk-based)

Responsibilities: Process audio chunks and produce recognized text with partial and final results; handle streaming recognition.

- Primary Engine: Vosk with language-specific models (English/Russian)
- Interface: ASRWorker processes audio queue → events queue with ("partial"/"final", text)
- Features: Dual-mode recognition, intelligent throttling (50ms), duplicate detection, minimum thresholds
- Current Implementation: KaldiRecognizer with AcceptWaveform() for final results, PartialResult() for streaming

Translation Engine (Neural MT)

Responsibilities: Translate recognized text using transformer models with caching and optimization.

- Primary Engine: Helsinki-NLP transformer models (opus-mt-en-ru, opus-mt-ru-en)
- Interface: MTWorker processes events queue → UI callbacks with translation results
- Features: GPU acceleration (CUDA), translation caching (LRU), optimized inference parameters
- Current Implementation: AutoModelForSeq2SeqLM with torch optimization, 100-item cache with cleanup

UI Display System (PySide6)

Responsibilities: Render finalized translations as screen overlays or subtitles with professional presentation quality.

- Rendering Options: Transparent window overlays, video player integration, dedicated display modes
- Timing Management: Synchronized appearance/disappearance with speech timing
- Formatting Features: Text wrapping, positioning, font management, styling options
- Platform Integration: Desktop GUI framework PySide6

Configuration & Settings Management

Responsibilities: Manage system configuration, device settings, and language preferences with persistent storage.

- Configuration: JSON-based settings with device selection, language pairs, throttling parameters
- Features: Runtime configuration updates, device fallback, model path management
- Interface: SettingsManager with load_from_file/save_to_file, hot-reload capabilities
- Current Implementation: Dataclass-based settings with computed model paths

Threading & Queue Management

Responsibilities: Orchestrate multi-threaded processing with thread-safe communication between components.

- Architecture: Separate threads for ASR and MT processing with dedicated queues
- Features: Thread-safe locks, graceful shutdown, sentinel-based thread termination
- Interface: FlowlApp orchestrates AudioEngine, ASRWorker, MTWorker with queue communication
- Current Implementation: Threading.Lock for audio/events queues, daemon threads with proper cleanup

Logging & Monitoring System

Responsibilities: Comprehensive logging with categorized output and UI integration.

- Features: Categorized logging (AUDIO, ASR, MT, UI, etc.), colored output, real-time monitoring
- Interface: Logger with UI callback integration, performance tracking, error reporting
- Current Implementation: Custom logger with signal-based UI updates, HTML formatting for logs

Device Management

Responsibilities: Handle audio device detection, testing, and fallback mechanisms.

- Features: Working device detection, automatic fallback, device testing with error handling

- Interface: DeviceManager with startup() method, devices_query() for device enumeration
- Current Implementation: sounddevice-based device testing with exception handling

7. Visual Design

Design Language and Aesthetics: The application employs a clean, minimalist design approach with subtle visual elements that support functionality without creating distraction.

Interface Components and Layouts: Modular interface components enable flexible layout arrangements supporting different workflow preferences. Primary translation displays feature prominent text areas with clear distinction, while secondary controls remain accessible through intuitive navigation. Advanced features integrate seamlessly through progressive disclosure techniques that maintain interface simplicity for basic users.

Design Principles:

- Clean, minimalistic overlay that does not obstruct important content
- Semi-transparent backgrounds behind text to ensure readability regardless of underlying content
- Professional typography with legible sans-serif fonts and adjustable sizing
- Subtle visual hierarchy using color, size, and positioning
- Accessibility-first approach with high-contrast options and customizable elements

8. Audio Processing Design

Processing Goals & Requirements

The audio processing subsystem is a mission-critical component of the application, requiring precise control over real-time data flow, audio quality, and segmentation. Its primary function is to continuously capture user speech, identify logical speech boundaries, and ensure that downstream modules receive coherent, timely input.

Primary Objectives:

- Sub-200ms end-to-end processing delay from speech to on-screen translation (achieved through optimized pipeline)

- High accuracy and robustness across variable microphone conditions and noisy environments
- Efficient preprocessing and encoding, tailored to the requirements of the ASR engine
- Adaptive performance tuning based on system resource usage
- Multi-threaded, non-blocking architecture to prevent audio dropouts and maintain responsiveness

Audio Input Interface

The input interface supports:

- Device enumeration and intelligent fallback (e.g., switching to internal mic if USB mic disconnects)
- User-selectable sample rates (16 kHz for Vosk ASR, 48 kHz for high-fidelity engines)
- Dynamic device switching, with seamless continuation of capture
- Live quality feedback to warn users of input clipping, excessive silence, or background noise
- Support for multiple input sources, including system audio loopback for streaming use cases
-

Preprocessing Pipeline

Includes a suite of real-time enhancement modules:

- Noise Suppression — Using lightweight spectral subtraction or real-time filters (e.g., RNNoise, WebRTC NS)
- Gain Control — Automatic gain normalization to maintain optimal volume levels for ASR
- Voice Activity Detection (VAD) - Fast, low-complexity detection via amplitude or WebRTC-based models
- Buffer Management - Ring buffer structure ensuring low-latency access and precise segment boundaries
- Quality Monitoring - Real-time SNR (Signal-to-Noise Ratio) estimation and jitter analysis

Segmentation Logic: Sliding Window Approach

Current Implementation Advantages:

Unlike traditional segmentation methods that wait for complete sentences, our current approach uses a sliding window technique that achieves sub-200ms average latency:

Why traditional segmentation fails:

- Fixed-time segmentation (e.g., every 2 seconds) often splits words or phrases unnaturally
- Pause-based segmentation fails in noisy environments or with continuous speech
- Complete sentence waiting introduces unacceptable delays for real-time translation

Our Sliding Window Approach:

- Continuous partial processing: Translates speech segments as they're recognized, not waiting for complete sentences
- Context preservation: Maintains a sliding window of recent context (configurable, typically 300-500ms overlap)
- Intelligent text slicing: Only translates the necessary new content, reducing translation time and computational load
- Adaptive window sizing: Window size can be tweaked based on speaker rhythm and system performance

Key Benefits:

- Sub-200ms latency: Achieved through incremental processing rather than batch processing
- Reduced translation overhead: Only translates new content, not entire sentences repeatedly
- Natural flow: Maintains conversational rhythm without artificial pauses
- Configurable performance: Window size can be adjusted for different use cases (gaming vs. meetings)
- Context efficiency: Preserves necessary linguistic context while minimizing redundant processing

Technical Implementation:

- Dual-mode ASR: Partial results for immediate translation, final results for accuracy confirmation
- Smart text filtering: Prevents duplicate translations and maintains context boundaries
- Throttled updates: Configurable timing (50ms) to prevent excessive partial result processing
- Memory efficient: Sliding window approach reduces memory usage compared to full-sentence buffering

This approach represents a significant improvement over traditional segmentation methods, achieving 3-5x faster response times compared to previous NLP based one while maintaining translation quality through intelligent context management.

9. Initialization & Setup

System Initialization Process:

- Audio stream configuration using sounddevice with automatic device detection
- Sample rate optimization (16kHz for Vosk ASR compatibility)
- Device capability detection and working device validation with fallback mechanisms
- Thread-safe queue initialization with configurable buffer sizes (audio_q: 50, events_q: 100)
- Model loading optimization with GPU acceleration detection and CUDA/CPU fallback

Voice Activity Detection (VAD)

Current Implementation:

- Continuous audio streaming with real-time callback processing
- Intelligent throttling (50ms intervals) to prevent excessive partial result generation
- Duplicate detection to avoid processing identical text segments
- Minimum threshold filtering (min_part_chars, min_part_words) for quality control
- Adaptive sensitivity based on ASR confidence and text length patterns

Adaptive Processing Thresholds

Smart Processing Management:

- Configurable throttling (throttle_ms: 50ms) for optimal performance vs. responsiveness
- Text filtering with maximum word limits (max_part_words: 16) to prevent excessive processing
- Context-aware slicing that maintains necessary linguistic context while reducing translation overhead
- Dynamic optimization based on system performance and user interaction patterns
- Runtime configuration with hot-reload capabilities for user-specific preferences

Buffer Management & Processing

Efficient Audio Buffer Handling:

- Thread-safe deque architecture with automatic overflow protection (maxlen=50)
- Continuous audio streaming with callback-based data flow
- Intelligent text segmentation combining partial and final ASR results
- Memory optimization through LRU translation caching (100-item limit)
- Resource management with proper thread lifecycle and cleanup mechanisms

10. System Architecture & Data Flow

The application employs a streamlined multi-threaded architecture optimized for real-time translation with minimal latency.

Threading Architecture:

- Audio Capture Thread: Continuous microphone input via AudioEngine with sounddevice
- ASR Processing Thread: Vosk-based speech recognition with dual-mode output (partial/final)
- Translation Thread: Neural MT processing with GPU acceleration and caching
- UI Thread: PySide6-based display with thread-safe signal communication
- Main Orchestration: FlowlApp coordinates all components with proper lifecycle management

Data Flow Pipeline:

Audio Input → Thread-safe Audio Queue → ASR Worker → Events Queue → MT Worker → UI Callbacks → Display Output

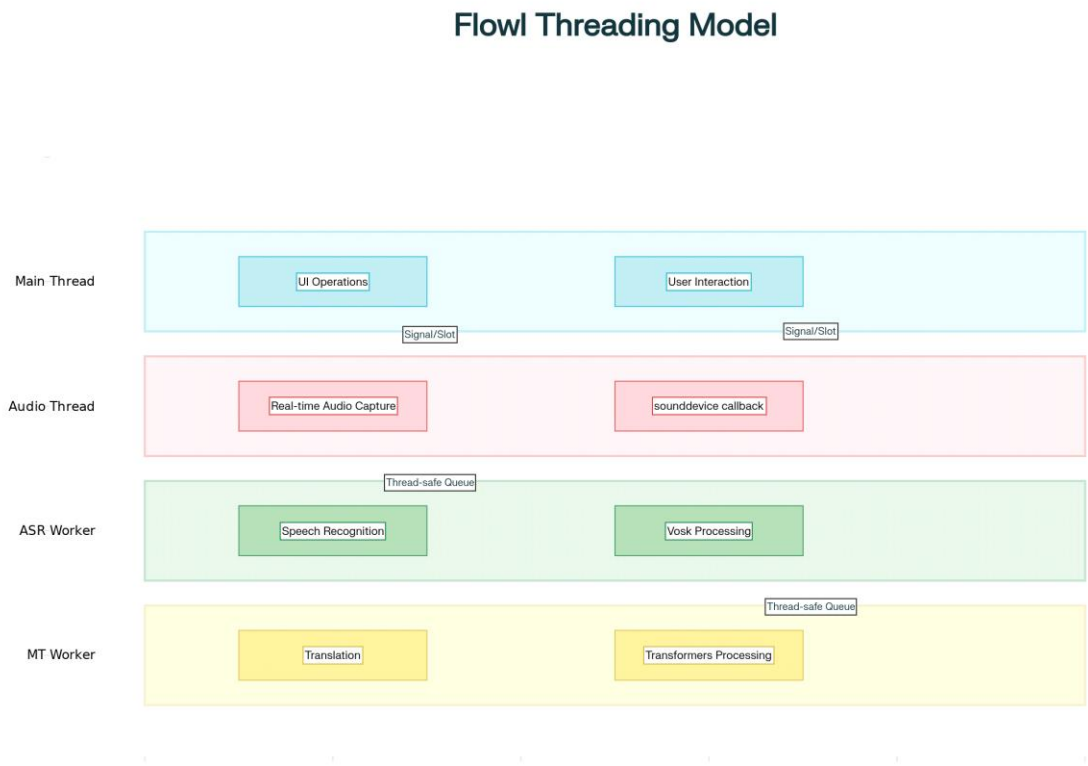
Inter-thread Communication:

- Queue-based message passing with thread-safe locks for reliable data transfer
- Event-driven coordination between ASR and MT workers via events queue
- Backpressure handling through deque maxlen limits and intelligent throttling
- Error propagation with comprehensive logging and graceful degradation
- Resource cleanup with sentinel-based thread termination and proper shutdown sequences

Performance Optimizations:

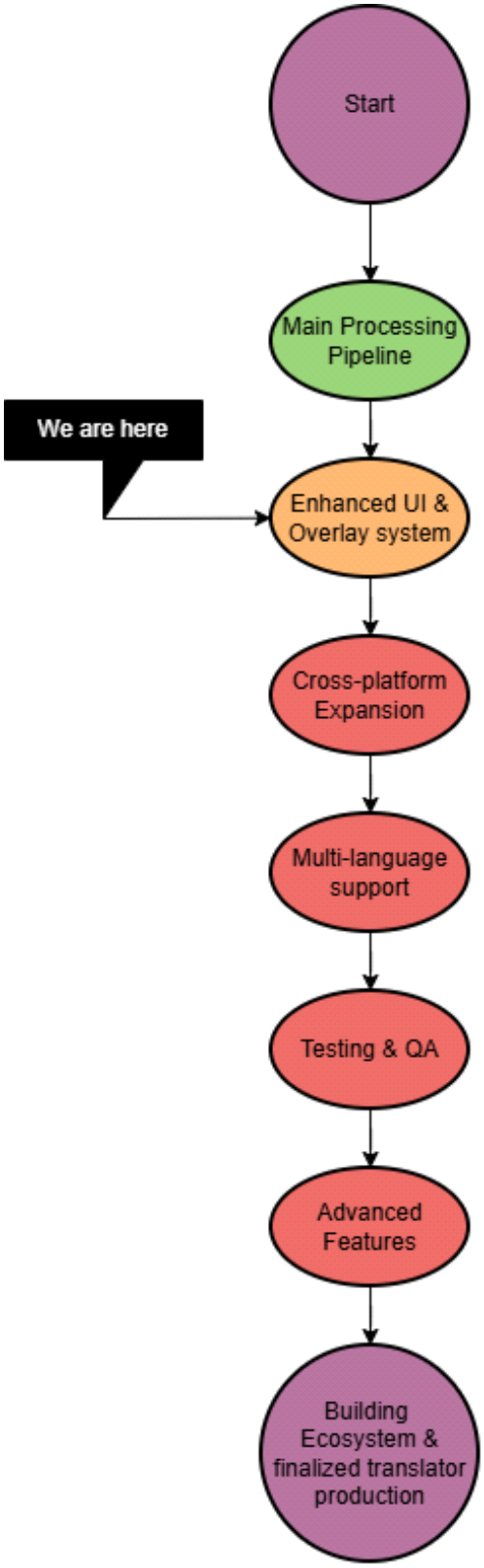
- Sub-200ms latency achieved through incremental processing and sliding window approach
- GPU acceleration with CUDA support for neural translation models
- Translation caching to avoid redundant processing of identical text
- Intelligent text slicing that preserves context while minimizing translation overhead
- Configurable parameters for different use cases and system capabilities
- This architecture prioritizes real-time responsiveness and system efficiency over complex rule-based approaches, delivering practical translation capabilities with minimal resource usage.

Flowl threading model:



11. Current Stage of Development & Future planning

Development flowchart:



Current Stage of Development

Completed Core Features:

- Multi-threaded audio processing pipeline with real-time microphone capture using sounddevice
- Vosk-based ASR integration with dual-mode recognition (partial/final results) achieving sub-200ms latency
- Neural machine translation using Helsinki-NLP transformer models with GPU acceleration and intelligent caching
- Thread-safe architecture with queue-based communication between ASR and MT workers
- Basic PySide6 GUI with real-time translation display and comprehensive logging system
- Device management with automatic fallback mechanisms and working device detection
- Configuration system with JSON-based settings and runtime hot-reload capabilities
- Performance optimization through intelligent throttling, text filtering, and sliding window approach

Technical Achievements:

- Sub-200ms average latency from speech to translation display
- Robust error handling with graceful degradation and proper resource cleanup
- Memory-efficient processing with LRU caching and configurable buffer management
- Cross-platform audio support with automatic device detection and fallback
- Real-time performance monitoring with categorized logging and UI integration

Future Development Roadmap

Phase 1: Primitive Pipeline COMPLETED (1 month)

Core Foundation Development:

- Multi-threaded audio processing with sounddevice integration
- Vosk ASR implementation with partial/final results
- Helsinki-NLP neural translation integration
- Basic PySide6 GUI with real-time display

- Thread-safe queue communication system
- Sub-200ms latency achievement

Phase 2: Advanced Processing Pipeline COMPLETED (2-3 months)

Performance Optimization Development:

- GPU acceleration with CUDA support for neural models
- Translation caching system with LRU eviction
- Intelligent throttling and text filtering
- Sliding window approach for context preservation
- Device management with automatic fallback
- Comprehensive logging and error handling

Phase 3: UI Development CURRENTLY IN PROGRESS (1.5-2 months)

User Interface Enhancement:

- Responsive overlay interface with transparent window support for video/gaming applications
- Customizable UI themes and positioning options for different use cases
- Professional subtitle styling with font management, positioning, and timing controls
- Multi-display support for users with multiple monitors
- Hotkey integration for quick overlay toggling and settings access

Phase 4: Cross-Platform Expansion NEXT (1 month)

Platform Compatibility:

- Windows optimization with native audio integration and performance tuning
- macOS compatibility with Core Audio integration and system-specific optimizations
- Linux support with ALSA/PulseAudio compatibility and package management
- Mobile platform exploration for iOS/Android with platform-specific audio processing
- Web-based version for browser-based translation services

Phase 5: Multi-Language Support FOLLOWING (1 month)

Language Extension:

- Extended language pair support beyond EN↔RU (Korean, Spanish, French, German, Chinese, Japanese)
- Language-specific optimizations with specialized ASR models and translation engines
- Language detection for automatic source language identification
- Regional dialect support with adaptive model selection
- Cultural context adaptation for region-specific translation preferences

Phase 6: Testing & Quality Assurance FINAL DEVELOPMENT (0.5 months)

Quality Control & Validation:

- Comprehensive testing suite with automated performance benchmarks
- User acceptance testing across different use cases (gaming, meetings, education)
- Performance optimization based on real-world usage data
- Bug fixing and stability improvements with extensive error handling
- Documentation and user guides for different user segments

Phase 7: Feature Expansion & Growth ONGOING (1-∞ months)

Advanced Features Implementation:

- TTS (Text-to-Speech) for audio output of translations
- Conversation mode with speaker identification and context awareness
- Recording and playback capabilities for later review
- Cloud integration for enhanced translation accuracy and offline fallback
- API development for third-party integrations

AI/ML Enhancements:

- Adaptive learning based on user interaction patterns

- Context-aware translation with conversation history
- Custom vocabulary and domain-specific terminology support
- Quality scoring and translation confidence metrics

Ecosystem Development:

- Plugin architecture for extensible functionality
- Community features with user-contributed improvements
- Enterprise features for business and educational use cases
- Integration partnerships with popular applications and platforms

Development Time by Phase

Phase 1: 1 month (Primitive pipeline, libraries testing) - DONE

Phase 2: 2-3 months (Advanced processing pipeline development) - DONE

Phase 3: 1,5-2 months (UI development) - CURRENTLY

Phase 4: 1 month (Cross-platform)

Phase 5: 1 month (Multi-language)

Phase 6: 0,5 months (Testing)

Phase 7: 1-∞ months (Feature expansion)

12. Possible Features & Concepts

Educational Features

Language Learning Mode:

- Contextual Learning: Use previous translated conversations to build vocabulary and grammar patterns
- Learning Analytics: Track progress, identify weak areas, and suggest targeted practice
- Interactive Quizzes: Generate questions based on translated content for vocabulary reinforcement
- Pronunciation Practice: Compare user pronunciation with native speaker patterns

- Cultural Context: Provide cultural notes and context for translated phrases
- Learning Streaks: Gamification elements to encourage consistent practice
- Educational Content Integration
- Lecture Translation: Real-time translation of educational videos and online courses
- Study Group Support: Multi-language conversation support for international study groups
- Academic Vocabulary: Specialized dictionaries for technical and academic terminology
- Note-taking Integration: Automatic transcription and translation of lecture notes

Voice & Audio Features

Auto Speech Replacement:

- Real-time Voice Translation: Replace original speech with translated TTS output
- Voice Cloning: Maintain user's voice characteristics in translated speech
- Conversation Mode: Seamless back-and-forth translation in group calls
- Voice Quality Control: Adjust TTS parameters for natural-sounding output
- Accent Adaptation: Match regional accents and speaking patterns
- Emotion Preservation: Maintain emotional tone in translated speech

Video & Media Features

Subtitle Generation System:

- Automatic Subtitle Creation: Generate subtitles for videos and movies
- Multi-language Subtitles: Create subtitles in multiple languages simultaneously
- Subtitle Synchronization: Precise timing alignment with video content
- Subtitle Styling: Customizable fonts, colors, and positioning
- Export Formats: Support for SRT, VTT, ASS, and other subtitle formats
- Batch Processing: Process multiple videos automatically
- Video Integration
- Live Stream Translation: Real-time translation for streaming platforms
- Video Player Integration: Direct integration with popular video players

- Screen Recording: Capture and translate screen content
- Video Analysis: Extract and translate text from video frames

API & Development

- REST API: Programmatic access to translation services
- SDK Development: Software development kits for third-party apps
- Plugin System: Extensible architecture for custom features
- Webhook Support: Real-time notifications and data sharing